Supplementary Material for: ECHO: Shared Execution of Graph Queries for LLM Agents

Junjie Yao
Software Engineering Institute, East China Normal University, Shanghai, China
junjie.yao@sei.ecnu.edu.cn

This document supplements the paper. Section S1 proves the exactness and work guarantees stated at the end of Section IV of the paper and the soundness and size of the masked index of Section IV-A; Section S2 lists the thresholds of Section V; Section S3 details the protocol of Section VI-A and of Exp-2 and Exp-3; Section S4 reports the query-size results of Exp-6 and further results of Exp-2 and Exp-4.

S1. Proofs

a) Exactness: Claim: for every member i of a merged set, the bag that the masked envelope plan returns for i equals $[\![q_i]\!]_G$.

Sketch. Fix i and keep the tuples whose mask contains i. The vertex, edge and depth layers and the refinement clear bit i only where no result of q_i can pass (see the soundness claim below). The extension step of Algorithm 1 keeps bit i exactly when the new vertex satisfies q_i's label, edge, predicate and distinctness tests, and the continue and output masks of Algorithm 2 keep it exactly when the trail fits q_i's hop range, label set and direction. The restricted expansion is therefore q_i's own enumeration in the shared order. The padded tuple of the masked OPTIONAL carries i exactly when i treats the subpattern as optional and no extension carries i, so the restricted outer join is q_i's, and the aggregation combines a partition of i's tuples per key. □

b) Shared work: Claim: in a fixed operator order, the work of a masked envelope plan is at most the sum of its members' work in that order, plus one mask operation per tuple and the construction of $\mathcal{C}_S$; when the members differ only in the hop ranges of one variable-length edge, the expansion of that edge does the work of the widest member.

Sketch. For the second statement, the continue mask extends a trail while some member's upper bound admits it, and the widest member needs every such trail. For the first, a tuple of the plan has a non-empty mask, so some member i produces the same partial binding alone; assigning it to the smallest such i maps the tuples of the plan injectively into those of the members, and likewise the edge expansions, which the plan performs only for members in the tuple's mask and, on $\mathcal{C}_S$, only along entries whose mask meets it. □

c) Soundness and size of $\mathcal{C}_S$: Claim: Algorithm 3 of the paper clears bit i only where no match of q_i uses the vertex or entry. With N_E stored entries, $\mathcal{C}_S$ takes $O\big(\sum_\kappa |D(\kappa)| + N_E\big)$ words and is built in $O\big(\sum_\kappa \sum_{v\in D(\kappa)} \deg_S(\kappa, v)\big)$ time plus one breadth-first search per distinct depth key, where $\deg_S(\kappa, v)$ is the number of data edges at v whose label some member requires at κ. For a single member this is the bound of the single-query index of CECI and DP-iso.

Sketch. Every cleared bit fails a condition that each match of q_i satisfies at that position: a vertex label or predicate in the vertex layer, the edge type and direction in the edge layer, a neighbour along an edge of q_i in the refinement, or a target within u_i hops in the depth layer. Each domain vertex and entry stores one mask. The vertex and edge layers read each domain vertex's adjacency once per incident edge that some member has, and each refinement pass reads every stored entry once, with one word operation per mask word. □

S2. Implementation Thresholds

The edge and depth layers are built only when their estimated construction work is at most a tenth of the work the merge saves, $\sum_{q\in S} \widehat{W}(q) - \widehat{W}(S)$, and only for classes of at most 10^5 candidates and target classes of at most 200 vertices; otherwise the operators apply the same masked tests on the adjacency lists of G.

We use $\rho = 0.25$ on every graph. For a set whose members differ in topology the threshold is $\max(B_S, 12\,\widehat{W}(S))$, and a wall-clock watchdog of $\max(60\,\mathrm{s}, 100\gamma B_S)$, with $\gamma = 1\,\mu\mathrm{s}$ per unit, stops a plan between counter checks.

Execution thresholds. A set with $B_S < 2 \cdot 10^4$ work units executes directly on adjacency lists, without constructing $\mathcal{C}_S$. A topology-changing merge must have an estimated cost at most ρ times the estimated cost of separate execution and save at least 200 estimated work units. Rounds with $\sum_q \widehat{W}(q) < 300$ units run per candidate. Point lookups with equal edge type, direction and endpoint label share one index lookup and adjacency scan.

Let $\widehat{W}_1$ be the first-clause estimate of Section V-B of the paper. A prefix group H requires $\rho \sum_{q\in H} \widehat{W}(q) \geq 2 \cdot 10^4$, at least $5 \cdot 10^3$ units of first-clause work, and $\max_{q\in H} \widehat{W}_1(q) < 0.98 \sum_{q\in H} \widehat{W}_1(q)$, so one member does not dominate the group. It replaces $|H|$ first-clause evaluations with one and runs to completion without a budget.

S3. Experimental Protocol and Full Results

a) Methods: MASC [1] (SIGMOD'26) and BatchEnum+ [2] (ICDE'24) are ported onto the subgraph-matching framework of Sun and Luo [3]; neither has public code. The MASC port implements shared signatures, visit-once enumeration and block reuse; the BatchEnum+ port implements Algorithm 4. On LDBC, they jointly evaluate pure-pattern and path candidates, respectively, and evaluate the others individually. GuP [4] (SIGMOD'23) also runs on MASC's graphs, following MASC's comparison. VLPQ [5] (ICDE'24) uses the official Graphflow implementation per candidate; its JVM time is a reference alongside the work comparison. *Per-candidate* uses the base filter and enumerator for pure patterns and ECHO's executor with one candidate per plan otherwise.

b) Metrics: We report round time, work, peak resident memory, and completion. Work counts edge expansions and intermediate tuples, instrumented in the common base framework and in VLPQ's official implementation. The time cap is 3,600 s per round and, following BatchEnum, 10,000 s per query set on its graphs; capped runs enter ratios at the cap. Completion is the fraction of candidate runs (candidates times repetitions) that finish within it. Efficiency ratios compare methods on the same batch and core, with the numerator stated in each table or figure. We report their geometric mean, 95% bootstrap interval and the share of batches ECHO wins. Scalability experiments vary batch size, graph size or query size while holding the other dimensions fixed.

c) Implementation: All runs use one server with two AMD EPYC 9654 processors and 1 TB of memory (Ubuntu 22.04); ECHO and the two ports are compiled into the base framework [3] with GCC 11.4 at `-O3`. Every run is single-threaded and pinned to one core with a fixed memory limit; the methods of a round run interleaved on the same core, five times (per-candidate evaluation once on three SF10 rounds of Exp-1). Separate correctness checks compare candidate result bags against per-candidate evaluation.

d) Published-workload protocols: On MASC's graphs we assign 100 random vertex labels where a graph has none; each query is a connected induced subgraph obtained by a random walk, 15 vertices by default; a batch holds 30 queries and a setting 300, and the overlap $\beta \in \{0, 3, 6, 9, 12\}$ makes families of five queries share a planted core that grows with β. On BatchEnum's graphs we follow its query-similarity experiment (its Definition 4.5 and Figure 7) and its set-size experiment from 100 to 500 queries (its Figure 8) for hop-constrained s–t queries with $\gamma = 0.5$; a set mixes p copies of one query ($k = 7$) with $100-p$ queries between disjoint small components ($k \in \{4, 5\}$). On LiveJournal we run the five path queries of VLPQ's Figure 11, restricted to edge label 0 of its 80-label protocol, which the queries traverse; both methods return directed walks, as the official implementation does, and we report the work of the shared and separate expansions because VLPQ runs on the JVM.

e) LDBC run provenance: SF1 runs map the graph from a shared read-only cache. At SF10, 703 of each method's 1,000 round runs come from the preceding build, which loads the graph in each process and differs only in how an OPTIONAL edge between two bound vertices is expanded; the 21 rounds this change affects and the unfinished runs (289 per method) use the current build, and 8 runs per method are unrecorded.

f) Candidate rounds: The LDBC rounds come from a frozen pool of 8,000 candidates for 200 questions, five rounds of $K = 8$ per question (1,408 distinct texts), sampled from a locally served Qwen model with CypherBench's zero-shot prompt and executed as generated; SF10 uses the official substitution parameters. 635 SF1 rounds contain at least two distinct CQ^+ candidates and enter the table, with 200 SF10 rounds. Candidates outside CQ^+ (3,375 on LDBC) run through the same Fallback in every method.

S4. Additional Results

The query-size results of Exp-6, the edit shares of Exp-1 and the β curves of Exp-2 are in the paper (Table VI, Figures 7 and 8).

a) MASC graphs without overlap: Table I gives the $\beta = 0$ setting of Exp-2 with memory and completion.

b) VLPQ, one query per round: In the earlier single-query configuration of the LiveJournal workload (VLPQ Figure 11, one thread), each round holds one query, so ECHO forms no shared plan and runs the earlier Fallback evaluator; both return the same 1.6 to 102.6 million walks. The official code takes 0.47 to 1.49 s for limits 3 to 7; VLPQ takes 0.044 to 0.48 times ECHO's time (0.081 [0.078,0.083] overall) and 0.34 times its work, counted by instrumenting the official code. The gap widens at limits 6 and 7, where VLPQ's jump operator replaces three hops with one hash join.

References

[1] Z. Zhang, S. Yang, W. Zheng, Q. Zhou, and C. Hong, "Beyond maximum common subgraph: A framework maximizing shared computation for multi-query subgraph matching," *Proc. ACM Manag. Data*, vol. 4, no. 3, pp. 1–27, 2026.

[2] L. Yuan, K. Hao, X. Lin, and W. Zhang, "Batch hop-constrained s-t simple path query processing in large graphs," in *ICDE*, 2024.

[3] S. Sun and Q. Luo, "In-memory subgraph matching: An in-depth study," in *SIGMOD*, 2020, pp. 1083–1098.

[4] J. Arai, Y. Fujiwara, and M. Onizuka, "GuP: Fast subgraph matching by guard-based pruning," *Proc. ACM Manag. Data*, vol. 1, no. 2, pp. 1–26, 2023.

[5] M. Li, P. Peng, Z. Hu *et al.*, "Variable-length path query evaluation based on worst-case optimal joins," in *ICDE*, 2024, pp. 3311–3323.

AI Acknowledgment

The text of all sections was drafted and revised with Claude (Anthropic, through Claude Code) and OpenAI Codex from outlines and instructions given by the authors, who specified the content, checked every statement against the code and the measurements, and edited the final wording. The same tools wrote, from the authors' designs, the TikZ and matplotlib sources of the figures, the implementation of the method and the baselines' ports, and the scripts that ran the experiments and produced the reported numbers. The problem formulation, the algorithms, the experimental design and the interpretation of the results are the authors'. The LLM-generated query candidates

TABLE I

PURE PATTERNS ON THE MASC GRAPHS (EXP-2) AT OVERLAP 0, 15-VERTEX QUERIES, 30 QUERIES PER BATCH. RATIO: METHOD TIME OVER ECHO TIME, GEOMETRIC MEAN OVER FIVE BATCHES AND FIVE INTERLEAVED RUNS WITH A 95% BOOTSTRAP INTERVAL (ABOVE ONE FAVORS ECHO); MEMORY: PEAK RESIDENT SET SIZE IN MB; COMPLETION: PERCENTAGE OF QUERIES WHOSE MATCHES WERE ALL FOUND. THE MASC PORT IS 1.03–1.06× FASTER THAN GUP HERE AND SLOWER AT $|V_q| = 32$ AND 64 (EXP-6 OF THE PAPER); MASC REPORTS A LARGER ADVANTAGE OVER SINGLE-QUERY METHODS ON DBLP AND WORDNET (FIG. 12 OF [1], PLOTTED ONLY).

	Figeys			DBLP			WordNet		
Method	Ratio	Mem.	Compl.	Ratio	Mem.	Compl.	Ratio	Mem.	Compl.
GuP	0.71 [0.62,0.80]	12	100	1.34 [1.33,1.35]	174	100	1.25 [1.24,1.27]	108	100
Per-candidate	0.60 [0.53,0.68]	3	100	1.14 [1.09,1.18]	140	100	1.07 [1.05,1.09]	75	100
MASC (port)	0.67 [0.58,0.78]	3	100	1.29 [1.27,1.32]	138	100	1.21 [1.18,1.25]	75	100
ECHO	1	6	100	1	248	100	1	146	100

that form the evaluated workloads are experimental inputs described in Section VI-A of the paper, not AI-generated content of this paper.